

CS 5350/6350: Machine Learning Fall 2024

Homework 4

Handed out: 12 Nov, 2024
Due date: 11:59pm, 26 Nov, 2024

- You are welcome to talk to other members of the class about the homework. I am more concerned that you understand the underlying concepts. However, you should write down your own solution. Please keep the class collaboration policy in mind.
- Feel free to discuss the homework with the instructor or the TAs.
- Your written solutions should be brief and clear. You do not need to include original problem descriptions in your solutions. You need to show your work, not just the final answer, but you do *not* need to write it in gory detail. Your assignment should be **no more than 15 pages**. Every extra page will cost a point.
- Handwritten solutions will not be accepted.
- *Your code should run on the CADE machines. You should include a shell script, `run.sh`, that will execute your code in the CADE environment. Your code should produce similar output to what you include in your report.*
You are responsible for ensuring that the grader can execute the code using only the included script. If you are using an esoteric programming language, you should make sure that its runtime is available on CADE.
- Please do not hand in binary files! We will *not* grade binary submissions.
- The homework is due by **midnight of the due date**. Please submit the homework on Canvas.

1 Paper Problems [40 points + 10 bonus]

1. [9 points] The learning of soft SVMs is formulated as the following optimization problem,

$$\begin{aligned} \min_{\mathbf{w}, b, \{\xi_i\}} \quad & \frac{1}{2} \mathbf{w}^\top \mathbf{w} + C \sum_i \xi_i \\ \text{s.t. } \forall 1 \leq i \leq N, \quad & y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \\ & \xi_i \geq 0 \end{aligned}$$

where N is the number of the training examples. As we discussed in the class, the slack variables $\{\xi_i\}$ are introduced to allow the training examples to break into the margin so that we can learn a linear classifier even when the data is not linearly separable.

- (a) [3 point] What values ξ_i can take when the training example $\mathbf{x}_i$ breaks into the margin?
 ξ_i takes values such that $0 < \xi_i < 1$, as this indicates $\mathbf{x}_i$ lies inside the margin but is correctly classified.
- (b) [3 point] What values ξ_i can take when the training example $\mathbf{x}_i$ stays on or outside the margin?
 $\xi_i = 0$ when $\mathbf{x}_i$ is on or outside the margin and correctly classified.
- (c) [3 point] Why do we incorporate the term $C \cdot \sum_i \xi_i$ in the objective function? What will happen if we throw out this term?
 The term $C \cdot \sum_i \xi_i$ penalizes violations of the margin. If this term is omitted, the model might allow unlimited margin violations, potentially leading to overfitting or poor generalization.
2. [6 points] Write down the dual optimization problem for soft SVMs. Please clearly indicate the constraints, and explain how it is derived. (Note: do NOT directly copy slides content, write down your own understanding.)

The dual form of the optimization problem is:

$$\begin{aligned} \max_{\alpha_i} \quad & \sum_{i=1}^N \alpha_i - \frac{1}{2} \sum_{i,j=1}^N \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j, \\ \text{s.t.} \quad & \sum_{i=1}^N \alpha_i y_i = 0, \\ & 0 \leq \alpha_i \leq C, \forall i = 1, \dots, N. \end{aligned}$$

This is derived by introducing Lagrange multipliers for the constraints and optimizing the Lagrangian function.

3. [10 points] Continue with the dual form. Suppose after the training procedure, you have obtained the optimal parameters.
- (a) [4 points] What parameter values can indicate if an example stays outside the margin?
 If $\alpha_i = 0$, the corresponding example $\mathbf{x}_i$ lies outside the margin.
- (b) [6 points] if we want to find out which training examples just sit on the margin (neither inside nor outside), what shall we do? Note you are not allowed to examine if the functional margin (i.e., $y_i(\mathbf{w}^\top \mathbf{x}_i + b)$) is 1.
 Examples just on the margin satisfy $0 < \alpha_i < C$. These are the support vectors, and their contributions are critical for determining the decision boundary.

4. [6 points] How can we use the kernel trick to enable SVMs to perform nonlinear classification? What is the corresponding optimization problem?

To enable nonlinear classification, replace $\mathbf{x}_i^\top \mathbf{x}_j$ with a kernel function $K(\mathbf{x}_i, \mathbf{x}_j)$. The optimization problem remains the same except for this substitution. The kernel function implicitly maps the data into a higher-dimensional space.

5. [9 points] Suppose we have the training dataset shown in Table 1. We want to learn a SVM classifier. We initialize all the model parameters with 0. We set the learning rates for the first three steps to $\{0.01, 0.005, 0.0025\}$ and hyperparameter $C = 1$. Please list the sub-gradients of the SVM objective w.r.t the model parameters for the first three steps, when using the stochastic sub-gradient descent algorithm.

x_1	x_2	x_3	y
0.5	-1	0.3	1
-1	-2	-2	-1
1.5	0.2	-2.5	1

Table 1: Dataset

- **Step 1: Example 1** ($\mathbf{x} = [0.5, -1, 0.3], y = 1$) The sub-gradient is computed as:

$$\Delta \mathbf{w} = \mathbf{w} - Cy\mathbf{x} = [0, 0, 0] - 1 \cdot 1 \cdot [0.5, -1, 0.3] = [-0.5, 1, -0.3].$$

For the bias b :

$$\Delta b = -Cy = -1 \cdot 1 = -1.$$

- **Step 2: Example 2** ($\mathbf{x} = [-1, -2, -2], y = -1$) The sub-gradient is computed as:

$$\Delta \mathbf{w} = \mathbf{w} - Cy\mathbf{x} = [-0.5, 1, -0.3] - 1 \cdot -1 \cdot [-1, -2, -2] = [0.5, 3, 1.7].$$

For the bias b :

$$\Delta b = -Cy = -1 \cdot -1 = 1.$$

- **Step 3: Example 3** ($\mathbf{x} = [1.5, 0.2, -2.5], y = 1$) The sub-gradient is computed as:

$$\Delta \mathbf{w} = \mathbf{w} - Cy\mathbf{x} = [0.5, 3, 1.7] - 1 \cdot 1 \cdot [1.5, 0.2, -2.5] = [-1, 2.8, 4.2].$$

For the bias b :

$$\Delta b = -Cy = -1 \cdot 1 = -1.$$

Summary of Steps:

- **Step 1:** $\Delta \mathbf{w} = [-0.5, 1, -0.3], \Delta b = -1$
- **Step 2:** $\Delta \mathbf{w} = [0.5, 3, 1.7], \Delta b = 1$

- **Step 3:** $\Delta \mathbf{w} = [-1, 2.8, 4.2]$, $\Delta b = -1$

6. [Bonus][10 points] Let us derive a dual form for Perceptron. Recall, in each step of Perceptron, we add to the current weights $\mathbf{w}$ (including the bias parameter) $y_i \mathbf{x}_i$ for some misclassified example $(\mathbf{x}_i, y_i)$. We initialize $\mathbf{w}$ with $\mathbf{0}$. So, instead of updating $\mathbf{w}$, we can maintain for each training example i a mistake count c_i — the number of times the data point $(\mathbf{x}_i, y_i)$ has been misclassified.

- [2 points] Given the mistake counts of all the training examples, $\{c_1, \dots, c_N\}$, how can we recover $\mathbf{w}$? How can we make predictions with these mistake counts? The weight vector $\mathbf{w}$ can be expressed in terms of the mistake counts $\{c_i\}$:

$$\mathbf{w} = \sum_{i=1}^N c_i y_i \mathbf{x}_i.$$

Predictions for a given input $\mathbf{x}$ can be made as:

$$\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x}) = \text{sign} \left(\sum_{i=1}^N c_i y_i (\mathbf{x}_i^\top \mathbf{x}) \right).$$

- [3 points] Can you develop an algorithm that uses mistake counts to learn the Perceptron? Please list the pseudo code.

Below is the pseudocode for the perceptron using mistake counts:

Algorithm 1 Perceptron with Mistake Counts

```

1: Initialize mistake counts  $c_i \leftarrow 0$  for all  $i = 1, 2, \dots, N$ 
2: repeat
3:   for all training examples  $(\mathbf{x}_i, y_i)$  do
4:     Compute prediction:
```

$$\hat{y} \leftarrow \text{sign} \left(\sum_{j=1}^N c_j y_j (\mathbf{x}_j^\top \mathbf{x}_i) \right)$$

```

5:   if  $\hat{y} \neq y_i$  then
6:     Increment mistake count:  $c_i \leftarrow c_i + 1$ 
7:   end if
8: end for
9: until Convergence or a fixed number of epochs
10: Return the mistake counts  $\{c_i\}$ .
```

- [5 points] Can you apply the kernel trick to develop an nonlinear Perceptron? If so, how do you conduct classification? Can you give the pseudo code for learning this kernel Perceptron?

By applying the kernel trick, we replace the dot product $\mathbf{x}_j^\top \mathbf{x}_i$ with a kernel function $K(\mathbf{x}_j, \mathbf{x}_i)$. The pseudocode for the kernel perceptron is as follows:

Algorithm 2 Kernel Perceptron

```
1: Initialize mistake counts  $c_i \leftarrow 0$  for all  $i = 1, 2, \dots, N$ 
2: repeat
3:   for all training examples  $(\mathbf{x}_i, y_i)$  do
4:     Compute prediction:  $\hat{y} \leftarrow \text{sign} \left( \sum_{j=1}^N c_j y_j K(\mathbf{x}_j, \mathbf{x}_i) \right)$ 
5:     if  $\hat{y} \neq y_i$  then
6:       Increment mistake count:  $c_i \leftarrow c_i + 1$ 
7:     end if
8:   end for
9: until Convergence or a fixed number of epochs
10: Return the mistake counts  $\{c_i\}$ .
```

Predictions for new examples $\mathbf{x}$ are computed as:

$$\hat{y} = \text{sign} \left(\sum_{i=1}^N c_i y_i K(\mathbf{x}_i, \mathbf{x}) \right).$$

2 Practice [60 points + 10 bonus]

1. [2 Points] Update your machine learning library. Please check in your implementation of Perceptron, voted Perceptron and average Perceptron algorithms. Remember last time you created the folders “Perceptron”. You can commit your code into the corresponding folders now. Please also supplement README.md with concise descriptions about how to use your code to run these algorithms (how to call the command, set the parameters, etc). Please create a new folder “SVM” in the same level as these folders.
2. [28 points] We will first implement SVM in the primal domain with stochastic sub-gradient descent. We will reuse the dataset for Perceptron implementation, namely, “bank-note.zip” in Canvas. The features and labels are listed in the file “classification/data-desc.txt”. The training data are stored in the file “classification/train.csv”, consisting of 872 examples. The test data are stored in “classification/test.csv”, and comprise of 500 examples. In both the training and test datasets, feature values and labels are separated by commas. Set the maximum epochs T to 100. Don’t forget to shuffle the training examples at the start of each epoch. Use the curve of the objective function (along with the number of updates) to diagnosis the convergence. Try the hyperparameter C from $\{\frac{100}{873}, \frac{500}{873}, \frac{700}{873}\}$. Don’t forget to convert the labels to be in $\{1, -1\}$.
 - (a) [12 points] Use the schedule of learning rate: $\gamma_t = \frac{\gamma_0}{1 + \frac{20}{a}t}$. Please tune $\gamma_0 > 0$ and $a > 0$ to ensure convergence. For each setting of C , report your training and test error.
 - (b) [12 points] Use the schedule $\gamma_t = \frac{\gamma_0}{1+t}$. Report the training and test error for each setting of C .

- (c) [6 points] For each C , report the differences between the model parameters learned from the two learning rate schedules, as well as the differences between the training/test errors. What can you conclude?
3. [30 points] Now let us implement SVM in the dual domain. We use the same dataset, “bank-note.zip”. You can utilize existing constrained optimization libraries. For Python, we recommend using “`scipy.optimize.minimize`”, and you can learn how to use this API from the document at <https://docs.scipy.org/doc/scipy-0.19.0/reference/generated/scipy.optimize.minimize.html>. We recommend using SLSQP to incorporate the equality constraints. For Matlab, we recommend using the internal function “`fmincon`”; the document and examples are given at <https://www.mathworks.com/help/optim/ug/fmincon.html>. For R, we recommend using the “`nloptr`” package with detailed documentation at <https://cran.r-project.org/web/packages/nloptr/nloptr.pdf>.
- (a) [10 points] First, run your dual SVM learning algorithm with C in $\{\frac{100}{873}, \frac{500}{873}, \frac{700}{873}\}$. Recover the feature weights $\mathbf{w}$ and the bias b . Compare with the parameters learned with stochastic sub-gradient descent in the primal domain (in Problem 2) and the same settings of C , what can you observe? What do you conclude and why? Note that if your code calculates the objective function with a double loop, the optimization can be quite slow. To accelerate, consider writing down the objective in terms of the matrix and vector operations, and treat the Lagrange multipliers that we want to optimize as a vector! Recall, we have discussed about it in our class.
- (b) [15 points] Now, use Gaussian kernel in the dual form to implement the nonlinear SVM. Note that you need to modify both the objective function and the prediction. The Gaussian kernel is defined as follows:

$$k(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{\gamma}\right).$$

- Test γ from $\{0.1, 0.5, 1, 5, 100\}$ and the hyperparameter C from $\{\frac{100}{873}, \frac{500}{873}, \frac{700}{873}\}$. List the training and test errors for the combinations of all the γ and C values. What is the best combination? Compared with linear SVM with the same settings of C , what do you observe? What do you conclude and why?
- (c) [5 points] Following (b), for each setting of γ and C , list the number of support vectors. When $C = \frac{500}{873}$, report the number of overlapped support vectors between consecutive values of γ , i.e., how many support vectors are the same for $\gamma = 0.01$ and $\gamma = 0.1$; how many are the same for $\gamma = 0.1$ and $\gamma = 0.5$, etc. What do you observe and conclude? Why?
- (d) **[Bonus]** [10 points] Implement the kernel Perceptron algorithm you developed in Problem 8 (Section 1). Use Gaussian kernel and test γ from $\{0.1, 0.5, 1, 5, 100\}$. List the training and test errors accordingly. Compared with the nonlinear SVM, what do you observe? what do you conclude and why?